



Kunal Kumar — Portfolio

Apple Developer Academy @ BINUS, Bali · Cohort 2027 · Tech track

Computer Science, class of 2028 · Frontend Engineering Intern, Emergent (YC S24) · Bengaluru [Portfolio](#) · [GitHub](#) · [LinkedIn](#) · kunalsain0324@gmail.com

The thing I keep building

Five projects, one obsession: **systems that refuse to invent.**

Software fails in one direction almost every time. It guesses, and it delivers the guess in the voice it uses for facts. A reconciliation engine picks one of four possible answers and posts it. A search bar returns something rather than nothing. A memory tool ranks probabilistically, so the same question gives you two different pasts. The system stays quiet, the totals still balance, and nobody finds out until it is expensive.

I build the other kind. I've now done it five times, and I only noticed the pattern afterwards.

- **ATTEST** refuses to post ₹99.5 lakh it cannot prove — and reports the refusal as the product, not as a failure rate.
- **Recall** refuses to rank probabilistically, and keeps no model in the production path, so your own memory can't return two different versions of last Tuesday.
- **Minutes** names the one thing in it that isn't real, in the README, before the reviewer can find it.
- **Upstream** is 29 patches judged by people who owe me nothing — the only work here I couldn't grade in my own favour.
- **Kunal HQ** awards me points only from verified data, so I can't lie to it. Which is the only reason I still believe it.

The through-line isn't caution. It's that **an honest "I don't know" has to be engineered** — deliberately, against the grain of the easier build — and almost nobody bothers.

That's the developer I already am. The Academy is where I want to learn to put it into something a person holds in their hand.

The five projects

PROJECT	WHAT IT IS	TYPE	STACK
1 · 🧠 Recall	A local-first continuity OS. One keystroke restores the tabs, files and chats from your last working session. Seven layers, a shipped <code>.dmg</code> , nothing leaves the machine	Self-initiated · Solo	Python · PyQt6 · FastAPI · ChromaDB · Tauri
2 · 📄 ATTEST	A financial control layer that verifies AI-proposed actions before they can move money. ₹0.00 wrongly posted across 500 held-out settlements	Self-initiated · Solo	Python · Rust
3 · 🗨️ Minutes	A meeting workspace — interactive transcripts, AI notes, FTS5 search — rebuilt from the product outward rather than the landing page inward	Work assignment · Solo	Next.js 15 · React 19 · FastAPI · SQLite
4 · 🌱 Upstream	29 patches merged into 35 open-source projects, reviewed and carried by the people who maintain them	Self-initiated · Solo, in public	Python · JS/TS · Fortran
5 · 🧩 Kunal HQ	A personal life OS — two-second capture from any terminal, a morning agent that assembles the day, and a game layer that cannot be cheated	Self-initiated · Solo	Shell · Markdown vault · Claude agent · launchd

Each page answers the six things the Academy asks for: what it is, whether it was self-initiated, my role, the impact I made, what I learned, and where to see it.

How to read this

Ordered by how much of me is in them, not by how impressive the stack sounds.

Recall is the deepest — 1,003 files, seven composed layers, and a shipped macOS app that honestly reports its own degraded mode. **ATTEST** is where the engineering standard is highest: real money, held-out seeds fixed in advance, and a benchmark table that names a competitor more precise than mine. **Minutes** is the

only assignment here, and the only one where I had to execute someone else's brief and someone else's design language. **Upstream** is the only work strangers graded before you saw it. **Kunal HQ** is last because it explains the other four.

Also built, not submitted here

Five is the maximum. These didn't get a page:

PDFChat ([live](#) · [code](#)) — page-cited answers, or an explicit refusal when the document doesn't support the question. **Support Triage** ([code](#)) — code owns the control flow, the LLM is a constrained analyst; 132/132 evaluation tests passing. **Typeahead** ([code](#)) — 120k real queries, a 3-node Redis ring, latency printed in the UI. **HTTP server from scratch** — HTTP/1.1 on raw sockets, thread-pooled, with path-traversal and Host-header defences. **Watershed** — a multi-agent RL arena, GRPO over 100k+ self-play episodes, where alliances formed on their own. **Web Agent** — perceive → decide → act, picking the right form out of nine by role matching rather than hardcoded selectors.

Thirty-two repositories in total, and this portfolio itself: hand-written HTML, CSS and vanilla JS. No framework, no build step, one page, one script.

Recall

Self-initiated · Solo · Python · PyQt6 · FastAPI · ChromaDB · Tauri · Chrome extension

[Live](#) · [Code](#) · [Docs](#)

The Broken System

Someone taps you on the shoulder at 3pm. When you sit back down, everything is still there — the file, the tab, the branch. And none of it helps.

Because you didn't lose the files. You lost **continuity**: the state of mind that held them together. You knew why that PDF was open. Now you don't. So you rebuild it by hand, badly, and the climb back costs more than the interruption did.

The tools we have all assume the wrong thing. Spotlight, Windows Search and `grep` assume **you remember a string**. A cognitive restart never asks “which file contains the word `tokenizer`.” It asks *what was I doing, which conversation was that in, where did I leave the implementation*.

Nothing on the machine answers those questions. Every tool answers the one question you are least able to ask.

This is not a search problem. It is a memory problem. And memory has structure that search throws away.

Three Kinds of Forgetting

You never forget one thing. You forget at three different scales at once, and a system that answers only one of them leaves you stranded.

 THE MOMENT	 THE SESSION	 THE MICRO-CONTEXT
A single thing you touched	A block of time you worked in	The topic you were actually chewing on
<i>“that arxiv page, yesterday”</i>	<i>“that evening I spent on reward shaping”</i>	<i>“the four visits that were all the same thought”</i>
Recoverable by keyword — barely	No keyword exists for it	No keyword exists for it
Spotlight can find this	Spotlight cannot	Spotlight cannot
Gap: it's the least useful of the three	Gap: the unit you actually schedule around	Gap: the unit a human actually remembers

These are three different questions, and one ranked list of filenames answers none of them properly. So Recall returns all three, in one list, from one query.

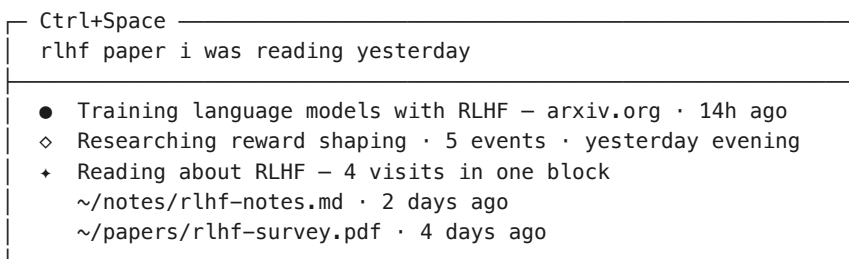
How the System Works

Seven layers in a single process, each composing on the one below it:

Events	per-day JSONL in ~/.recall/events/	"what did I touch?"
Sessions	~30-minute activity groupings	"what was I doing then?"
Micro-contexts	topic-coherent splits of a session	"what was I chewing on?"
Resurfacing	the quiet idle-launcher digest	"what should I notice?"
Threads	persistent topic identity over weeks	"what's the shape of my work?"
Evolution	chronological phases inside a thread	"how did this change?"
Recovery	resumable work, one-click restoration	"what do I open right now?"

A Chrome extension — 200 lines — posts page visits, searches and chat sessions to a local API at 127.0.0.1:4545. watchdog watches the folders you choose; save a file and it's reindexed within ten seconds. Semantic file search runs all-MiniLM-L6-v2 over a ChromaDB index persisted to ~/.recall/index/.

Then press **Ctrl + Space** and type a fragment.



Moment, session and micro-context, in that order, from one half-remembered phrase.

Open it with an **empty** query and it does the thing the product exists for: a quiet *Continue where you left off* card, **capped at four items**, no badges and no feed. Click one and the files, tabs and chats come back **in the order that re-grounds the work** — not the order you happened to open them.

You step back in. You don't rebuild.

The Design Decision — Why there is no LLM in the production path

The instinct is obvious: dump every event into a vector store, put a model on top, let it summarise what you were doing. It demos beautifully. It's the wrong call, on two counts.

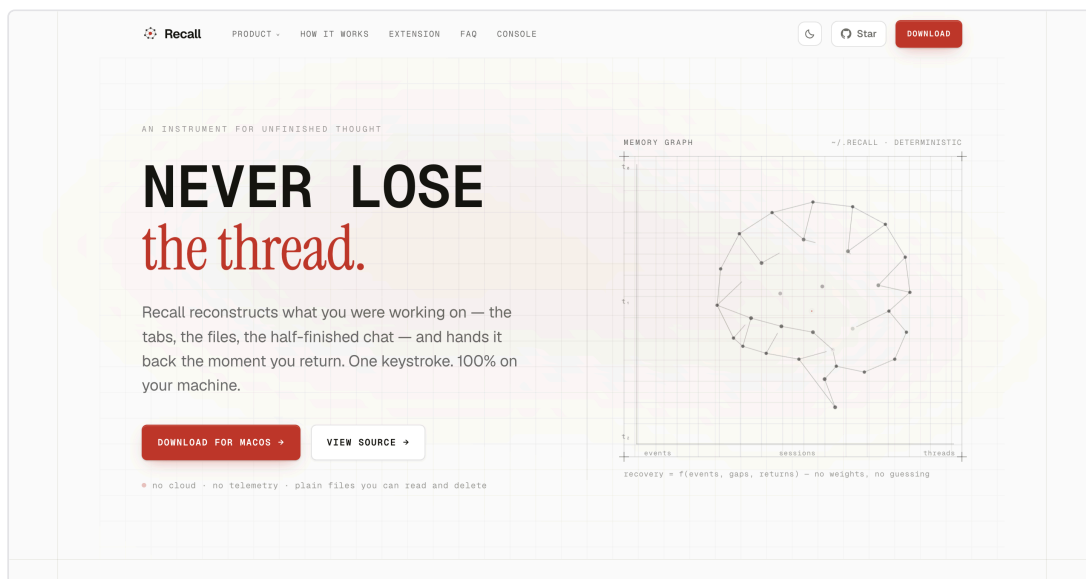
Determinism. The same events in must give the same output out. If a probabilistic ranker sits between you and your own memory, you can ask the same question twice and get two different pasts. A memory tool that is *sometimes* right is worse than none, because you lose the ability to tell which one you're holding. Every layer of the engine is deterministic, and a **35-section smoke test** exists to enforce exactly that contract.

Privacy you can check rather than be promised. This product reads your browsing, your files and your conversations — the most invasive permission set software can ask for. “We take your privacy seriously” is not an answer. So: no cloud, no account, no telemetry, no model call in the production path. Everything Recall knows about you lives in `~/recall/` as plain JSON and JSONL — **readable with `cat`**, **deletable with `rm`**. Delete the folder and it’s a complete reset. You don’t have to trust me; you can go look.

There’s a third reason, and it’s the one I’d defend hardest. **A chatbot answers questions. It does not restore state.** Recall never summarises your work or talks to you. You don’t want a conversation *about* your work — you want to be back inside it.

Those constraints removed the easy version of nearly every feature. That was the point.

The Interfaces



Recall’s shipped site and download.

Recall comes in two shapes, and it’s worth knowing which you’re looking at.

Run from source — one Python process: the PyQt6 launcher in `app/ui/`, plus the FastAPI service in `api/`, in a venv. Full stack, semantic file search included. Three hard layers with no leakage: `app/core/` is pure Python and imports no Qt; the UI talks to retrieval *only* through an HTTP client.

The shipped macOS app — a Tauri menu-bar launcher carrying a headless `recall-daemon` sidecar, which starts only if nothing already answers on `127.0.0.1:4545`. **~26 MB**, because the bundle leaves out PyQt6 and the embedding stack; semantic file search honestly reports *disabled* there, and the browser extension becomes the primary capture path.

Both speak the same `/v1/*` surface, so every claim holds for either.

The Proof

It ships. Not a repo — a `.dmg`, a documentation site at `docs.recall.computer`, a `SUPPORTED_PLATFORMS.md` that states where it doesn't work, a `SECURITY.md`, a changelog and a roadmap. **1,003 files.**

The bar I set before starting was a **three-minute install: five commands to a working launcher**. A continuity tool that takes an afternoon to configure will never be there on the afternoon you need it.

And the honest one: the shipped bundle reports semantic search as *disabled* rather than quietly returning worse results. A degraded mode that announces itself is a feature. A degraded mode that stays silent is a lie with a nice install size.

What I Learned

The hard problem was ranking, and ranking turned out to be editorial, not mathematical. The engine can surface hundreds of relevant events, and showing you hundreds of relevant events is identical to showing you nothing. Getting to *four items, no badges, no feed* meant repeatedly deciding what **not** to show — and each of those was a judgement about what a person needs in the ten seconds after sitting down, never a threshold I could tune. I went in thinking retrieval was the project. Restraint was the project.

Constraints adopted on purpose make the product; constraints discovered later make the backlog. Ruling out the cloud and ruling out the model cost me the easy build of almost every feature — and produced the only two claims about Recall a stranger can verify themselves in under a minute. I now write down what software is *not allowed to do* before I write down what it does.



ATTEST

Self-initiated · Solo · Python · Rust solver kernel · Held-out evaluation

[Code](#)

AI can decide. Your ledger can't guess.

The Broken System

A payment gateway does not pay a merchant order by order. It sends **one lump sum** covering many orders at once, minus its fees, and somebody has to establish what that credit was made of.

Where the records carry an order-level reference, that reconstruction is a join, and it is easy. The hard case is where they don't — a bank credit reconciled against a statement instead of a gateway report, a period whose recon is unavailable, an adjustment with no linked entity. There, composition has to be *derived* from amounts, fees and dates.

And here is the fact that turns a tedious problem into a dangerous one:

More than one set of orders can add up to the same credit exactly.

Those sets discharge **different customers**. Picking one is a guess. A guess marks the wrong customer as paid **while the books still balance** — so no alarm fires, no total looks wrong, and nobody finds out until it is expensive. The alternative today is a person in a spreadsheet.

Every AI finance tool being sold right now sits directly on this trap. It proposes an answer, the answer is plausible, and plausible is indistinguishable from correct until the customer calls.

This is not an accuracy problem. It is an **authority** problem — the question is not who is right, but who is allowed to be believed.

Two Settlements, Two Outcomes

Both are real settlements from the held-out evaluation. Every figure is regenerated from `benchmark/results.json` rather than typed, and both are pinned by a test so the data cannot move underneath them.

	SETL_000233	SETL_000225
event	₹6,523.53	₹23,922.07
advisor	proposed a capture batch	proposed a capture batch
verifier	2,328 → 4 → 1 · PROVEN, residual ₹0.00	2,328 → 23 → 4 · AMBIGUOUS, four valid explanations
policy	expected loss ₹247.82 < ₹250 review cost	not eligible at any price
ledger	AUTO-POST — balanced entry written	HOLD — no financial action

The advisor said the same thing about both. On the second, its proposal sits inside exactly one of the four valid explanations — so a system that listened to it would post ₹23,922.07 today, confidently.

Across the held-out panel, that advisor is right **27 times in 63**. A 43% opinion is not evidence. So ATTEST reports the ₹12,630.27 that is settled *whichever* explanation is right, holds the ₹30,107.39 that isn't, and hands the operator the one field that ends the argument.

How the System Works

Four stages, and the separation between them is the entire design:

ADVISOR	proposes investigative signals – which records to look at
VERIFIER	independently reconstructs and proves the explanation
POLICY	decides whether a proven result is safe to automate
LEDGER	records only permitted execution

The advisor is advisory only. It cannot authorise financial action. It is handed identifiers, names and dates — and never an amount. The verifier recomputes the explanation from the source records **without reading what the advisor said**. The ledger accepts nothing the verifier has not independently re-derived.

Three properties make that separation real rather than rhetorical:

Exact money. Integer paise end to end. No float ever touches an amount, so the rounding tolerance is a *derived bound* rather than someone's guess at an epsilon.

Independent verification. A 35-line checker re-derives every proof from the source records and **shares no code** with the search that produced it. The asymmetry is deliberate: a bug in the search costs recall; it cannot post a wrong entry.

A policy gate, not a threshold. Automation happens where the *measured* error rate for that class of result, priced at its 95% upper bound, costs less than a human check. Change what a review is worth and the boundary moves on its own. Nobody re-tunes a magic number.

The Design Decision — Why the refusal is the deliverable

Every instinct in this space says: raise coverage, resolve more, ship a number that goes up. I built the opposite metric and put it on the front page.

SAFETY			
proof precision	0.952	right when it claims sure	
false proof rate	0.80%	← the number that moves money	
MONEY			
processed	₹1,02,04,411.89		
auto-posted	₹2,52,431.44		
protected	₹99,51,980.45	refused, deliberately	
wrongly auto-posted	₹0.00		

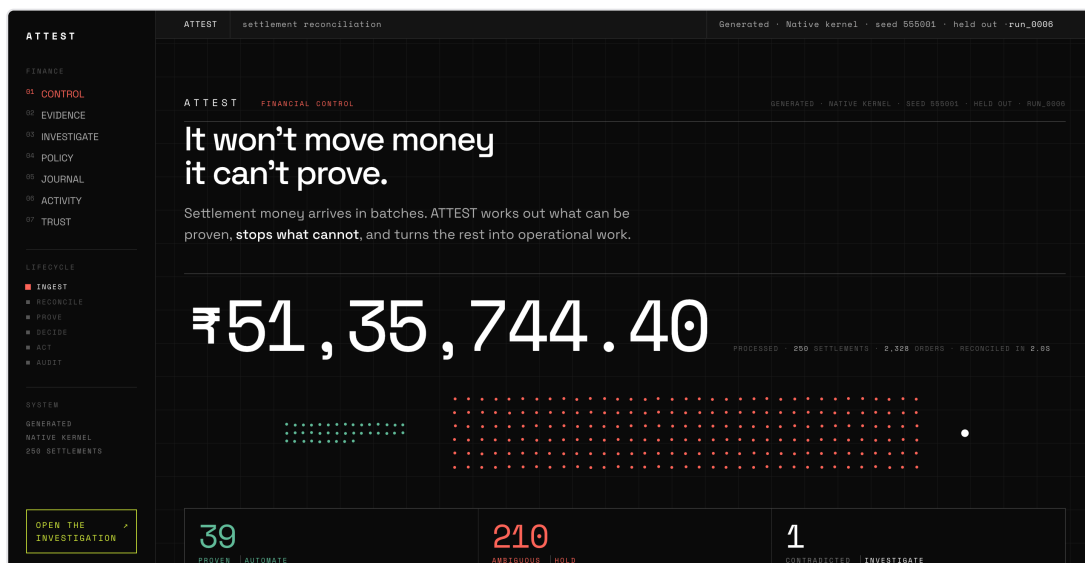
82.4% of settlements are correctly refused. Presented naively that reads as an 82% failure rate. It is the reason the system is deployable.

Which forces a distinction I had to build the whole architecture to be able to make: **a false proof and a wrongly posted entry are different things, and the gap between them is the product.** Over 500 held-out settlements the engine offered 84 proofs and 4 were wrong. **None of the 4 was posted** — the policy priced them into REVIEW or BLOCK. Of the 33 settlements that auto-posted, **33 were exact, and ₹0.00 moved against the wrong account.**

The Interface

The product surface is a control view over one run: how many settlements were processed, how many were proven and eligible to automate, how many are held as ambiguous, and how many are contradicted. On held-out seed 555001 that reads **39 proven, 210 held, 1 contradicted** out of 250.

The design decision inside the interface matches the one outside it. The held items are not an error list tucked below the fold — they are the main column, each carrying the one field that would resolve it. The operator's job is not to check the machine's work. It is to answer the specific question the machine could not.



The ATTEST control view on held-out seed 555001 — 250 settlements reconciled: 39 proven and eligible to automate, 210 ambiguous and held, 1 contradicted.

Held-out evaluation seed 555001, on the native kernel. Every figure is read from the run.

The Proof

Calibration and evaluation seeds are disjoint by construction. Against three reference matchers, on identical data and identical scoring:

matcher	coverage	decided	wrong	false proof	pair prec
attest	16.0%	84	4	4.8%	95.9%
exact_only	4.4%	22	0	0.0%	100.0%
fuzzy	3.6%	30	12	40.0%	60.0%
greedy	4.6%	462	439	95.0%	16.5%

500 settlements over seeds [555001, 999983]

Read the second row. **exact_only is more precise than ATTEST** — 100% against 95.9%. That sits in the README rather than being quietly dropped, because a benchmark you curated in your own favour is not evidence of anything. It buys one quarter of the coverage, which is the actual trade, and saying so is the only reason the other three rows mean anything.

Full methodology and metric definitions are in `docs/EVALUATION.md`. The seed panel is fixed in advance so the goalposts cannot move afterwards.

What I Learned

Integrity is a shape, not an intention. I could not simply decide to be careful. Refusing safely required the verifier to be *structurally incapable* of hearing the advisor, the money layer to be *structurally incapable* of rounding, and the automation boundary to be derived from measured error rather than chosen by me. Every one of those is an architecture decision made months before the first refusal ever fired. Build integrity into the shape of the system, or it's a promise in a README.

Publishing the number that beats you is what makes the rest believable. Leaving `exact_only`'s 100% precision in that table cost me the clean story. It bought something better: a reader can check my worst comparison and discover I ran it myself, first. I've stopped asking "are my results good." The question is "did I go looking for the result that would embarrass me, and did I print it."



Minutes

Work assignment (SDE Fullstack take-home) · Solo · Next.js 15 · React 19 · TypeScript · FastAPI · SQLModel · SQLite FTS5 · Tailwind

Code

Set as an assignment, not self-initiated. I built all of it alone — frontend, backend, schema, search index, summariser, seed data and docs. Not affiliated with Fireflies.ai.

The Broken System

“Clone this product” is the easiest brief in the world to do badly.

You screenshot the marketing site, rebuild the hero section, wire up two buttons, and ship something that photographs well and collapses the moment anyone clicks. Every clone I’ve seen does this, and they all fail at the same place: they rebuild what the product *looks like* on day one instead of what it *does* on day four.

Because a meeting-notes product is not its landing page. It’s what happens when you have eleven recorded meetings, you half-remember someone committing to something about pricing, and you need to find it. Nobody re-watches an hour of video. Everything valuable in that product happens **after** the meeting ends.

So I set the bar somewhere I couldn’t fake it: **everything except the speech-to-text has to actually work.**

Three Users, Three Problems

The four-column shell exists because three different people open this product for three unrelated reasons.

 THE PERSON WHO WAS THERE	 THE PERSON WHO MISSED IT	 THE PERSON SEARCHING WEEKS LATER
Sat through the call, remembers the vibe, not the specifics	Wasn’t invited, has to catch up before their own meeting	Half-remembers a commitment, no idea which meeting it was in
Needs the decisions and action items without the hour	Needs the shape of the conversation, fast	Needs to find one sentence across every transcript in the workspace
Would never rewatch, would skim notes	Would never rewatch, would skim the transcript	Keyword is all they have
Problem: notes are the product, video is the archive	Problem: a transcript is a wall of text unless it’s navigable	Problem: LIKE '%pricing%' is not search

Three different jobs, three different surfaces, one database. That’s why the meeting view puts **AI notes on the left, the interactive transcript on the right and the player pinned to the bottom** — and why clicking a transcript line seeks the player. That single interaction is what turns a transcript from a wall of text into a document you can actually use.

How the System Works

A Next.js 15 App Router frontend over a FastAPI backend, with SQLite as the store and **FTS5** as a genuine full-text index — not a `LIKE` query wearing a costume.

The URL structure mirrors the real product deliberately, so anyone who has used Fireflies navigates this on muscle memory:

PATH	SCREEN
<code>/home</code>	Home dashboard, with the assistant panel
<code>/notebook/mine-shared · /all · /favorites · /autopilot</code>	Meetings library
<code>/view/{id}</code>	Meeting detail — notes, transcript, player
<code>/search · /tasks · /soundbites · /upload · /analytics</code>	Workspace surfaces

There is exactly **one deliberate divergence**: `/` is the public marketing page here, so the app home lives at `/home`. It's documented, because an undocumented divergence is just a bug.

Auth is enforced in `middleware.ts` **at the edge**, so a signed-out visitor is redirected before any app UI renders — rather than the usual pattern of shipping the whole dashboard to the browser and hiding it with a spinner.

And on first boot the backend creates the database, builds the FTS index, and **seeds seven complete meetings** with full transcripts, notes and action items. No extra step. A reviewer clones it, runs two commands, and lands in a populated product instead of an empty state.

What's Real and What Isn't

I'd rather be exact about this than impressive about it.

	STATUS
Real speech-to-text	Out of scope , stated up front. Transcripts are seeded or uploaded.
AI notes & action items	Real — a deterministic extractive summariser in the backend. Optionally Claude, with a key.
Search	Real — SQLite FTS5.
Data	Real — it persists. Every CRUD path works.
Auth	Real — edge-enforced in <code>middleware.ts</code> .

The first row is the important one. Claiming transcription and shipping a `setTimeout` would have been the normal move. Drawing the line in the README meant every other row could be taken at face value.

The Design Decision — Rebuilding the token set instead of matching screens

The instinct is to pipette colours off screenshots. It's the wrong call, and it fails in a specific way: it looks right in one screenshot and subtly wrong everywhere else, because a real product's UI isn't a set of colours. It's a **token system**, and the effect you're copying is consistency across twenty surfaces, not the surfaces themselves.

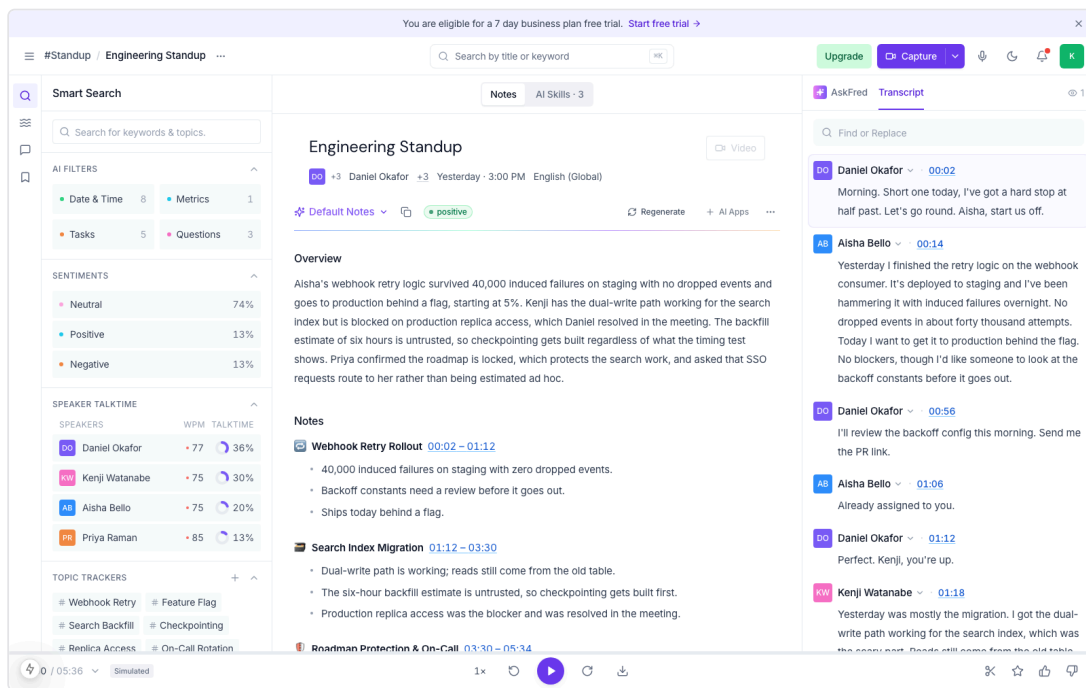
I spent the first stretch matching individual screens and got exactly that result — plausible in isolation, incoherent in aggregate.

So I threw it out and extracted the system instead. The Tailwind theme **is** Fireflies' own token set, written down as tokens and documented in `docs/design-notes.md`. Once the tokens were right, twenty-plus screens came out coherent without any of them being individually tuned.

The boring structural thing, done once, made every later screen correct for free.

The Interfaces

SCREEN	WHAT IT DOES
Meeting view	Notes, timestamped speaker-labelled transcript, player — the core loop
Meetings library	Channel sidebar; mine / shared / all / favorites / autopilot
Home dashboard	Workspace overview with the assistant panel
Search	FTS5 across every transcript in the workspace
Landing + login	The public marketing mirror, and auth into the seeded workspace



The meeting view — AI notes on the left, the interactive transcript on the right, the player pinned to the bottom, inside the four-column shell.

The meeting view: the core loop. Click a transcript line and the player seeks.

Full screenshots: [docs/screenshots](#).

The Proof

Two commands, from `git clone` to a working, populated meeting workspace — including the FTS index and seven seeded meetings. Reseeding is a documented flag, and `--generate` reruns the same summariser an upload would, which is the quickest way to see the gap between the authored fixtures and the machine-generated notes. I shipped the comparison that shows my own summariser's limits.

And the README opens with an explicit *Assumptions and scope* section. The missing transcription engine is either honest engineering or a broken promise, and the **only** difference is whether I wrote it down before the reviewer found it.

What I Learned

Fidelity is a property of the system, never of the surface. Matching screens produced something subtly wrong everywhere; extracting the token set produced something right everywhere, for less work. I now refuse to draw anything until I've found the system underneath it — and that has changed how I start every piece of UI work since.

Scope stated in advance is a feature; scope discovered by the reviewer is a defect. The same absent transcription engine can be either, and nothing about the code decides which. Naming the boundary cost me nothing and made every remaining claim credible. I now open a README with what the software doesn't do.



Self-initiated · Solo, but entirely in public · Python · JavaScript · TypeScript · Fortran

[Every upstream pull request →](#)

*Figures verified against the GitHub API on 11 September 2026: **29 merged, 44 open, across 35 repositories.***

The Broken System

Every other project in this portfolio is work I graded myself.

In a personal project, “done” is whatever I decide it is. I choose the problem, the standard, the definition of finished, and the moment to stop. That’s fine — it’s how you learn quickly — but it means the entire evaluation loop runs inside my own head, and a portfolio built only from that is a portfolio of self-assessment.

Upstream, “done” is when someone who has maintained a codebase for a decade — who did not ask for my patch, owes me nothing, and will be maintaining my code long after I’ve moved on — decides it’s good enough to carry.

That is a completely different standard, and it’s the only one here I couldn’t grade in my own favour.

It’s also the closest thing I’ve done to what a ten-month cohort actually is: **sustained collaboration with people who don’t work for you, in a codebase you didn’t write, under conventions you must learn before you’re entitled to an opinion.**

Where the Work Landed

PROJECT	MERGED	LANGUAGE	WHAT IT IS
sugarlabs/musicblocks	8	JavaScript	A music-and-programming microworld for children
django/djangoproject.com	3	Python	The site Django itself runs on
metabrainz/listenbrainz-server	3	Python + TypeScript	Open music listening data
plone/volto	2	JavaScript	The Plone 6 React frontend
ZeusLN/zeus	2	TypeScript	A Bitcoin Lightning wallet
fortran-lang/stdlib	1	Fortran	The Fortran standard library
fortran-lang/webpage	1	Python	Fortran's community site
openstreetmap/iD	1	JavaScript	The OpenStreetMap editor
plone/meta	1	Python	Plone's shared tooling
plone/plone.versioncheck	1	Python	Plone dependency tooling

Plus merged work in three smaller repositories. **Twelve approvals from ten different maintainers across seven organisations.**

The titles say more than the count does:

- musicblocks#8495 — *release the busy state so the widget cannot deadlock*
- musicblocks#8589 — *raise NanError for NaN in every operation that promises it*
- musicblocks#8588 — *give equal19 all nineteen of its ratios*
- plone.versioncheck#79 — *parse buildout files with zc.buildout's parser (instead of by hand)*
- stdlib#1207 — *point the fypp error message at the variable that works*
- iD#12794 — *detect goo.gl shortlinks as an incompatible source*
- zeus#4621 — *BBQr split/join invariants*
- listenbrainz#3856 — *read custom_name, not custome_name*

Almost none of these are features. They are a deadlock, a broken promise in an error contract, a missing ratio, a hand-rolled parser, a licensing trap for mapmakers, and a typo that silently disabled a feature. **This is what maintenance work actually looks like**, and it's the work that decides whether software still functions in five years.

The Design Decision — Going where I'd be uncomfortable

The efficient play is to find one friendly project and file twenty patches. The count climbs fast, the review culture becomes familiar, and you learn one codebase very well.

I deliberately spread across ecosystems that share nothing: a children's education tool, a Bitcoin wallet, a map editor, scientific data QC, Django's own website, and the Fortran standard library. **Four languages, six review cultures, six sets of conventions I had no say in.**

Fortran is the one I'd point at. I had no reason to know Fortran. Numerical scientists maintain `stdlib` to a standard where every patch is read very carefully, and my instincts from JavaScript were worth precisely nothing there. Landing something meant learning how that community argues about correctness *before* writing a line.

That's the skill I was actually practising. Not the language — the ability to arrive somewhere I have no standing and become useful anyway.

The Maintainers

The patches are the visible part. The work was learning what each maintainer needs, and it was different every time.

walterbender — Sugar Labs' org lead. Merged eight. Browser-tested one himself before merging, and pulled two other reviewers onto another. Wants you to act on automated review feedback rather than argue with it.

MonkeyDo — MetaBrainz. Two standing rules, stated plainly: keep the AI-usage disclaimer in the PR template, and run the tests locally before submitting. I discovered the backend suite cannot run on Apple Silicon — the `metabrainz/python` image is amd64-only across all 41 tags, and emulation segfaults. Rather than submit backend work I couldn't test, I moved to the frontend Jest suite, which runs locally. **All three merges came from that side.** The constraint chose the contribution.

sarahboyce — a Django Fellow, so the scarcest time here. Gives itemised reviews and expects every point answered. Pointed me at `django/django`'s own `SeleniumTestCaseBase` as the pattern to follow, which was better than what I'd written.

kaloudis — Zeus's lead. Ran the suite himself on my PR, then gave three concrete suggestions, the sharpest being negative tests for the QR-joining function — the one code path fed directly by untrusted camera input. All three are addressed.

rakshityadav1868 — not a maintainer, another contributor. Reviewed two of my musicblocks PRs carefully and **corrected me on #8594, where he was right and I was wrong.** That one belongs in the list. The correct response to being wrong in public is to say so on the thread and fix it; the correct thing after that is to read his pull requests as carefully as he read mine.

The Proof

Twenty-nine patches now live in software people use daily — a tool children learn music on, the editor behind OpenStreetMap, a Bitcoin wallet, and the Fortran standard library.

But the count isn't the evidence. **Ten maintainers who owed me nothing put their name on my work**, several of them more than once, and several now recognise it when it arrives. That's a relationship, and relationships are how open source actually gets built.

There's a rejection worth naming too: `laurent22/joplin` closed a patch of mine because the project only accepts PRs from long-term contributors. Not a quality judgement, and the right call for them — but it's the clearest lesson available that a merge is a decision about *trust*, not just about a diff.

What I Learned

A patch is a proposal to a specific person, and most of the work is understanding who. The same diff gets merged in one project and closed in another, and it is almost never about the code. MonkeyDo needed local test evidence. sarahboyce needed every review point answered individually. walterbender needed me to stop arguing with the bot. Reading what a maintainer actually needs — from their past reviews, before filing anything — improved my merge rate more than any improvement in my code did.

Being wrong in public is cheap; defending it is expensive. When rakshityadav1868 corrected me, the useful move was to concede in one comment and fix it. I'd assumed the cost of being visibly wrong in front of a whole project was high. It's close to zero, and it buys you a reviewer who takes you seriously. What's genuinely expensive is the thirty minutes I once spent defending something I already suspected was wrong.



Self-initiated · Solo · Shell · Markdown vault · Claude agent · macOS LaunchAgent · Hand-written HTML dashboard

Private repository, local demo. The only project here with no public URL — and it's in this portfolio anyway, because it's the one I built for a user I know everything about.

The Broken System

July 2026. A full-time internship with Slack pinging and tickets piling up, college deadlines, **three applications in flight**, a flight to book, and a personal life underneath all of it. Every open loop was being held in working memory, because there was nowhere else to put it.

The result wasn't laziness, and the data says so plainly: **120 LeetCode problems solved. GitHub days with 24 contributions.** The work was happening.

The result was **fog** — the mental static of too many open loops with no place to land. Every decision cost more than it should, because a background process was permanently running, checking whether something had been forgotten. Nothing was ever fully put down, so nothing was ever fully picked up.

The obvious diagnosis is a discipline problem and the obvious fix is a better habit. I'd tried that repeatedly. It doesn't work, for a reason worth naming:

Every capture tool I'd used charged its tax at exactly the wrong moment.

The thought arrives mid-task. Capturing it means opening an app, choosing a project, picking a tag, deciding whether it's a task or a note — and *deciding* is precisely the resource you had none of, which is why the thought was going to be lost.

So the tools built to reduce cognitive load were demanding cognitive load as their entry fee. Fog isn't cleared by thinking harder. It's cleared by writing things down, into a place that sorts, remembers, and shows you only what matters now.

Three Organs, Three Jobs

 THE VAULT	 THE AGENT	 THE PAGE
~/Life0S — plain markdown and JSON on my Mac	Claude, reading and writing the vault	dashboard.html , a render of the vault
One file per day, per project, per domain: journal/ , goals.md , projects/ , habits.md , fitness.md , applications.md , tickets.md , dates.md , notes.md	Infers intent from anything typed. Triage the capture queue. Pulls in the outside world each morning	Five surfaces: Today · Work · Life · Game · Manifest
Human-readable, greppable, portable, mine	Never interrogates. Hard-caps focus at three items	Opens, always, to exactly one thing
Law: archive, never delete · absolute dates only · the vault is the truth	Law: capture and processing are decoupled	Law: default-collapsed. Never show the whole house

The vault is the source of truth. Every page is only a render of it. If the agent and the dashboard both vanished tomorrow, the ten years of my life inside ~/Life0S would still open in any text editor.

How the System Works

The entire capture interface is one line, from whatever terminal I'm already in:

```
brain call maa about the flight
```

A shell function appends it to inbox/quick.md and **returns immediately**. No AI runs. Nothing loads. **Two seconds, zero judgment, sorted later.**

The intelligence happens afterwards, when the cost of thinking isn't being charged to me mid-task. The next agent run triages every queued line into the right file — tasks to the journal, ideas to ideas.md , dates to dates.md , protein to fitness.md — automatically.

And every morning at **8:23**, a macOS LaunchAgent fires a headless session that:

1. Drains and triages the capture queue
2. Pulls the world — weather, both GitHub accounts (open PRs, reviews, issues, contribution calendars via GraphQL), LeetCode counts and streak, calendar and reminders, Mac vitals, and the work-ticket ledger
3. Settles XP, audits badge conditions, rebuilds the learning graph
4. Rewrites the dashboard's data block and republishes it

Every source fails independently. A dead VPN never blocks the weather. The system degrades in pieces rather than collapsing.

I wake up and the day is already assembled.

The Design Decision — A game layer that cannot be cheated

There's an XP economy on top: check-in +10, focus item shipped +25, something learned +20, a LeetCode solve +30, a GitHub contribution +5, gym +40, protein target +20, the Sunday review +100. Levels unlock at $(n-1)^2 \times 100$. I'm **Fog Fighter, 570 XP**. Five of twelve badges unlocked — Genesis, Centurion, Medium Rare, Hard Mode, Green Wall.

The instinct is to let the user tick boxes. It's the wrong call, and it fails predictably.

Every gamified productivity app I've used eventually becomes a game *about the app*. You tick a box to move the number, the number stops corresponding to anything real, and within a fortnight you're maintaining a system instead of doing work. **A number that is cheap to move is a number you will quietly stop believing** — and you'll be right to.

So the rule is absolute: **XP is earned only from real, verified data. Never self-reported.** The LeetCode solve is read from LeetCode. The contribution is read from GitHub. The badge unlocks when the condition is true in the data, not when I say it is.

I cannot lie to it. That is the only reason I still believe it.

Which is the same principle as everything else in this portfolio, turned on myself. ATTEST refuses to post what it can't prove. Recall refuses to rank probabilistically. Kunal HQ won't award me points I didn't earn. I didn't plan that symmetry — I noticed it months later, looking at five projects side by side, and concluded that I only trust systems that are structurally incapable of flattering me.

The Interfaces

Five rooms, entered one at a time, never all at once.

Today (*always opens here*) — THE ONE THING banner, today's three focus items, a "waiting on you" digest, agenda and reminders, the player ring and daily quests. This *is* the day.

Work — tickets, including a **ghost work** column for the things that consume you without appearing anywhere. GitHub and Slack radar. The application pipeline, which currently has MATS and this Academy in it.

Life — habit heatmap and streaks, the learning graph, body metrics, the dreamlist, goals.

Game — trophy room, recent wins, the XP economy laid out honestly.

Manifest — the product poster. What this is, its principles, where it's going.

Inside each surface, cards **fold to one-line summaries** (**TICKETS** — 6 open · 3 ghost >). Today's cards open ready; every other room opens quiet, and the page remembers what I keep open.

Default-collapsed is law. The overwhelm this product exists to kill must never come from the product.

The look took **ten iterations** — bold, silver, beige, porcelain, sakura — before landing on serif-on-sand rose-gold with a drift of petals. A tool you open every morning of your life at 8am should not feel like a dashboard at work.

The Proof

It runs itself, every morning, on a schedule I don't touch, and it has for months. `hq` in any terminal opens it. The pinned tab reloads every ten minutes.

I use it every day. It runs my mornings. It's on v10.

That's the only metric I have and the only one that can't be gamed — I'm the entire user base, there's nobody to impress, abandonment would be instant and total, and I haven't abandoned it. `applications.md` is tracking this application while you read this page.

What I Learned

Adoption is decided at the moment of capture and nowhere else. My first version had good organisation — structure, categories, a sensible schema — and I used it for about a week. The version that stuck costs two seconds and asks nothing, with every organising decision deferred to a moment when I'm not busy. That generalised immediately, and it's now the first question I ask of any interface: **find the moment the user has least to give, and make sure that's the moment you ask for nothing.**

Building for exactly one user taught me more about product than building for a hypothetical many. There's nowhere to hide. I couldn't excuse a feature by imagining someone who thinks differently, because there is no one else, and I couldn't declare success from usage numbers, because I'd know I was forcing myself. Every honest signal came from noticing what I quietly stopped doing. It's the most direct feedback loop I've had, and it left me deeply suspicious of any feature I can't name a specific person for.